

Requirements Engineering 10 questions

1. What manual validation strategy is most appropriate for evaluating our pipeline's ability to match, rank, and categorize code clones (Type-1 through Type-4) across Language specific skills within the pinned GitSkills sample dataset?) **-Roy**
2. What are the precise distribution counts of repositories per primary programming language? and in specific C++ **-Roy**
3. Are there any ethical, privacy, or security concerns in relation to matching, ranking, and categorizing code clones? **-Roy**
4. What are some problems or concerns the sources do not establish? **-William**
5. What are some traps or common mistakes that can be done doing this? **-William**
6. According to the GitSkills preprint and dataset card, what exactly counts as a "skill" in this dataset, and what population of repositories was collected (time window, platforms, inclusion/exclusion criteria)? Please cite the section where this is defined. **-Rosh**
7. What tables or files does the GitSkills release contain, what is the primary unit of analysis in each, and how are they linked (e.g., which fields join a skill to its repository)? Please cite the dataset card or schema documentation, and note which dataset version this applies to. **-Rosh**
8. What limitations or known gaps do the GitSkills sources acknowledge (e.g., sampling bias, missing values, deduplication, authorship attribution), and what questions about the data can the current sources *not* answer that we would need to verify by inspecting the dataset directly? **-Rosh**
9. Is it feasible to detect and categorize Type-1 through Type-4 clones of SKILL.md files (and their bundled scripts) within the pinned GitSkills sample, restricted to C++ repositories? Specifically: which fields support each clone type, how many primary-representative skills would remain after filtering to C++, and is that enough for a meaningful analysis? **-Rosh**
10. Q10. How does GitSkills assign a programming language to skill? Is it only repos.language or bundled Java files can be identified through artifact_siblings? Roughly how many primary-representative skills in the pinned v1.0.0 sample come from Java repositories, and would a C++ vs. Java comparison of clone rates be feasible on the sample? **-Pranjal**

Question (Num)	Notebook answer (We Like)	Source cited by notebook	Team Interpretation	Follow-up or uncertainty
#1	Draw a stratified sample of 40–60 candidate clone pairs across different programming languages and copy-frequency tiers, then inspect them side-by-side using a strict taxonomy (Type-1 exact byte matches up to Type-4 semantic equivalences). Evaluate your	https://github.com/giuseppedestefanis/gitskills-sample	Create a sample population and verify our findings against the sample.	Precision@K, Mean Reciprocal Rank (MRR), and manual Markdown or code diff inspections. Will check the notebook

	pipeline's matching and ranking accuracy against this ground truth using metrics like Precision@K, Mean Reciprocal Rank (MRR), and manual Markdown or code diff inspections.			
#2	There are fewer than 8,094 C++ repositories	https://github.com/giuseppedestefanis/gitkills-sample	C++ is small enough of a scope for our project	What if there aren't enough repositories for C++?
#3	Static Analysis Only: Treat all skill content and scripts purely as text—never execute code found in the dataset. Security & Clone Precision: Do not label a clone candidate as functionally identical if it introduces altered scripts, network calls, or unexpected commands. Data Privacy Guardrails: Filter out potential PII or hardcoded credentials in raw skill files, and strictly avoid cross-referencing author hashes to deanonymize developers. License Awareness: Track how cloned skills cross repository license boundaries (e.g., permissive vs. copyleft). MSR Compliance: Include a dedicated "Ethical, Privacy, and Security Implications" section in your final report detailing these safety and open-science measures.	https://2027.msrconf.org/track/msr-2027-mining-challenge https://huggingface.co/datasets/mvaccargiu/gitkills	Seems pretty clear. We should do these throughout the project.	Reminder to add the ethics section to our repo
#4	Runtime Agent Execution and Performance: Neither dataset captures runtime execution logs, LLM context window contents, or actual pass/fail rates when AI agents execute skills or specs. Selection occurs probabilistically at run time without compiler or type-checker verification, meaning the datasets cannot establish how often skills or specs fail, cause model hallucinations, or generate faulty code Interactive Human-Agent Dialogue: SpecMine and GitSkills record static files, pull request diffs, and git commits. They do not establish multi-turn chat interactions, prompting iterations, or	https://2027.msrconf.org/track/msr-2027-mining-challenge	These are things to consider	Make sure to talk and communicate when doing the project

	how developers and agents collaboratively negotiate requirements during drafting.			
#5	Double-Counting Mass-Copied Skills (GitSkills): 50.5% of all SKILL.md file occurrences are verbatim copies. <i>Mistake:</i> Aggregating text or structural metrics across all rows in artifacts without filtering by <code>dedup_primary = 1</code> or grouping by <code>file_sha</code>. Doing so heavily overcounts widely reused templates. Querying Non-Primary Representative Rows for Content: Attributes like full text (content), parsed frontmatter, folder siblings (<code>artifact_siblings</code>), and sampled commit histories exist only for primary representatives (<code>dedup_primary = 1</code>). <i>Mistake:</i> Expecting non-primary copy rows to hold text or trying to join <code>artifact_siblings</code> to non-primary rows.	https://github.com/giuseppedestefanis/gitskills-sample#start-of-content https://github.com/shyamagarwal13/specmine-official#start-of-content	These are interesting mistakes that it found. I wonder if we would make these mistakes?	Lets make sure we don't fall into these traps
#6	A skill is a folder containing a SKILL.md file with natural-language instructions for an LLM agent, optionally bundled with scripts and reference files. The agent loads a skill at run time when it judges the task matches the skill's description. Anthropic introduced the format in October 2025. The dataset harvested every file matching SKILL.md via GitHub code search, then classified each by <code>location_class</code>: - canonical <code>.claude/skills/<name>/SKILL.md</code>: 9.8% - any <code>skills/</code> directory: 55.4%	https://arxiv.org/abs/2608.10906 https://huggingface.co/datasets/mvaccargiu/gitkills https://github.com/giuseppedestefanis/gitskills-sample https://doi.org/10.5281/zenodo.218	"Skill" is broad in the raw data. We must choose our population explicitly by filtering on <code>location_class</code> and <code>frontmatter_valid</code> , depending on whether we study spec-compliant skills or all SKILL.md-name	No explicit exclusion criteria were given beyond GitHub search limits. The exact query partitions live in the <code>mining_runs</code> table and in Table I of the preprint (arXiv:2608.10906), so we need to check both

	elsewhere: 34.8% 86.6% of distinct contents have valid YAML frontmatter. The population is public GitHub only, a snapshot from July 2026: 3,797,117 file occurrences, 1,877,981 distinct contents, 282,200 repos, and 195,841 accounts.	75637 https://2027.msrconf.org/track/msr-2027-mining-challenge	d files.	
#7	The release is v1.0.0: a SQLite DB (agent_skills_release.db, or agent_skills_sample.db for the 277 MB sample) plus a Parquet mirror on Hugging Face. It has four tables: artifacts: one row per SKILL.md occurrence, with primary key (repo_full_name, path). It has 3.8M rows (29,786 in the sample). Only rows with dedup_primary = 1 carry the content, frontmatter, and commit history. artifact_siblings: one row per bundled file or folder next to a representative skill. It has 7.26M rows (47,829 in the sample). repos: one row per repo, with primary key full_name. It has 282,200 rows (11,841 in the sample). mining_runs: one row per discovery run (7 rows), holding provenance metadata. The joins: artifacts.repo_full_name = repos.full_name (1:N) artifact_siblings joins to artifacts on (repo_full_name, artifact_path = path) file_sha groups	https://huggingface.co/datasets/mvaccargiu/gitskills https://github.com/giuseppede Stefanis/gitskills-sample https://doi.org/10.5281/zenodo.21875637 https://2027.msrconf.org/track/msr-2027-mining-challenge	Our unit of analysis for clone work is the distinct content, so we group by file_sha rather than by rows. Repo-level attributes like language and license come through the repos join.	We should confirm the join keys and row counts ourselves by querying agent_skills_sample.db, and pin v1.0.0 in RESEARCH_SPE C.md.

	verbatim copies across repos			
#8	Scope limits: The data covers public GitHub only and is a lower bound. Code search indexes default branches only, files under 384 KB, and repos with fewer than 500k files. Forks were included only if they had more stars than their parent, which left just 3 forks. Legacy files: The results include lowercase and non-standard files, some with history going back to 2014, before the spec existed. Deduplication: 50.5% of occurrences are verbatim copies. Only the primary representative is enriched, so copies may have different folders and histories that go unobserved. Commit history: It is only sampled (458,548 files, flagged by <code>history_fetched</code>) and follows the current path only, so renames distort <code>first_commit_at</code>. Folders and content: Sibling listings are capped (<code>composition_truncated</code>), content over 100 KB or non-text content is omitted, 0.9% of files were deleted before retrieval, and some skills are symlinks. Authorship: Authors are keyed hashes. There is no label for human-written vs LLM-generated skill text, and emails and names are masked. Unanswered by the sources: • How many non-canonical or invalid-frontmatter files are real agent instructions • Which script languages dominate the bundled siblings • How often renames occur • What share of skills are symlinks	https://huggingface.co/datasets/mvaccargiu/gitskills https://github.com/giuseppedestefanis/gitskills-sample	Deduplication and missing enrichment on copies directly affect clone detection. Exact (Type-1) clones are handled by <code>file_sha</code> , but near-duplicate detection must run on primary contents only. We also can't claim anything about who or what authored a skill.	Verify the limitation sections in the Dataset Card and Sample Repo, then run sample-DB queries on <code>location_classes = 'other'</code> with <code>frontmatter_valid = 0</code> and on symlink or short-content rows.
#9	Technically feasible, but a C++-only slice of the sample is small. Fields per clone type: Type-1 (exact) – group by artifacts. <code>file_sha</code> / artifact_siblings. <code>entry_sha</code> ; Type-2 (renamed) – normalize tokens and	https://github.com/giuseppedestefanis/gitskills-sample	The sample works for building and validating our	The "<100–200 skills" figure is the notebook's estimate, not a

	frontmatter in artifacts.content, name, description; Type-3 (near-miss) – edit distance / n-gram Jaccard / line diffs on artifacts.content and artifact_siblings.content; Type-4 (semantic) – embeddings + cosine similarity on content or description. Bundled-script text exists only for primary representatives (dedup_primary = 1), so sibling folders of copies in other repos aren't captured. The sample has 29,786 occurrences, 13,000 primary contents, and 11,841 repos. The sources do not publish a C++ count (C++ is outside the top 7 languages; Go is 7th with 8,094 repos), so it must be queried: artifacts JOIN repos ON repo_full_name = full_name WHERE dedup_primary = 1 AND language = 'C++'. The notebook estimates this likely yields fewer than 100–200 skills: enough for a pilot / pipeline validation, not for generalizable claims. Generalizing needs the full 41 GB release (1,877,981 primary skills, 282,200 repos).	sample https://huggingface.co/datasets/mvaccargiu/gitskills	clone pipeline, but C++-only results won't generalize. We should either widen the language scope or plan to run the final analysis on the full release.	documented fact. Run the C++ count query on agent_skills_sample.db to confirm. Note: the 8,094 in row #2 is Go's repo count, so C++ is simply below that, not a documented C++ count.
#10	Programming language is recorded only at the repository level (repos.language, GitHub's primary-language label), not per skill. Bundled Java files can be found separately by filtering artifact_siblings on entry_name (e.g., .java) or by searching sibling content (text files up to 100 KB). The sources give no Java count for the sample. Java is not in the top 7 languages of the full release, all of which are repository counts, from Python (71,801) down to Go (8,094). The sample is deterministic: the 13,000 distinct contents with the lowest content hashes, plus all their copies. A C++ vs. Java comparison is technically possible on the sample but statistically weak, and the notebook recommends the full release for reliable results.	[1] GitSkills preprint (arXiv:2608.10906), Sections I–II and Appendix A; [3][4] GitSkills sample repo README and Hugging Face dataset card (schema, sample construction, descriptive statistics); [6][7] join documentation.	"Java skill" has two possible definitions: a skill in a Java-primary repository, or a skill that bundles Java files. They won't match. A Java-primary repo can hold a skill with only Markdown, and a Python-primary repo can hold a skill with bundled .java files. We need to pick one definition, or report both, and state it as a data requirement. Also, the notebook defines "clone rate" as verbatim copies per content hash, which is only Type-1. Our Type-2 to Type-4 analysis needs its own definition.	The notebook's "<2.8%" is its own calculation, not a documented figure. $8,094 \div 282,200$ is about 2.87%, and it describes repositories, not skills. Its top-7 counts come from the full release, which confirms that the 8,094 in rows #2 and #9 is Go's full-release count, not a C++ figure. The Java and C++ skill counts for the sample are not documented anywhere, so we will verify them by querying agent_skills_sample.db in Colab, both by repository language and by bundled .java files. We also need to confirm that the preprint's cited Appendix A exists, since the paper is listed as 3 pages.

Group 5

Pranjal Oli

Roy Gartley

Roshkrishna Ranjith

William Mullane

Full Notebook Responses

Roy's Notebook

<https://notebook.google.com/notebook/37dcc7fc-7695-4b4f-b3e3-5c54e2e2313f>

Pranjal's Notebook

<https://notebook.google.com/notebook/a51848f8-9d5d-45bc-a244-7352960a7476>

Roshkrishna's Notebook

[Official MSR 2027 Mining Challenge: GitSkills and SpecMine Datasets - Gemini Notebook](#)